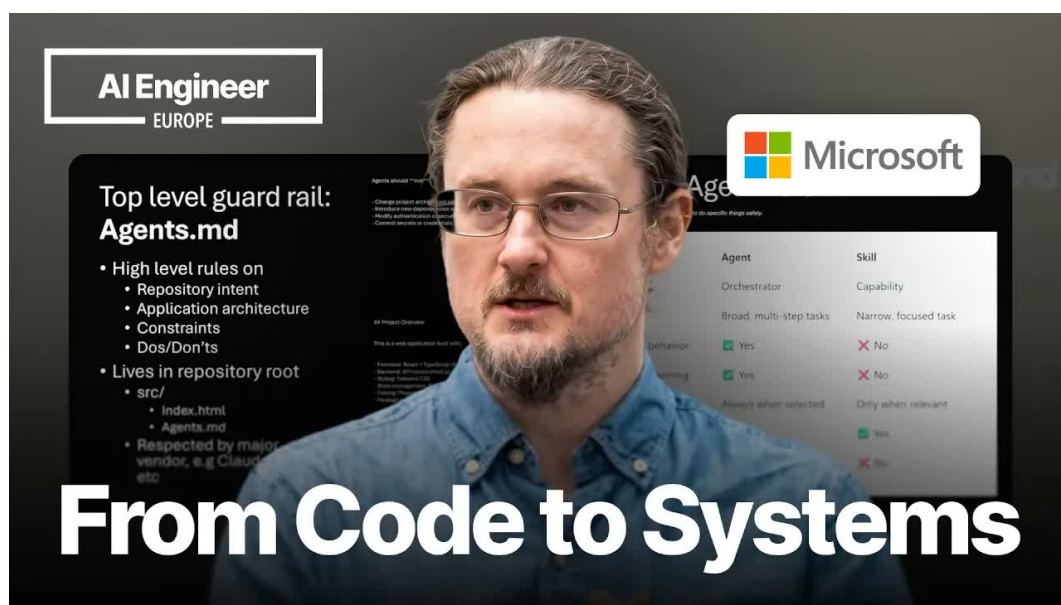


从写代码到设计系统：AI 代理时代的开发者角色与受控委派

基于 Chris Noring 演讲整理

2026 年 7 月 11 日



视频频道：AI Engineer

发布日期：2026 年 7 月 11 日

视频时长：23:05

视频链接：<https://www.youtube.com/watch?v=GdvKNwMcfD0>

来源限制：英文自动字幕完整；GoogleVideo CDN 在真实视频流下载阶段持续发生 TLS EOF，画面证据采用当前网络条件下最高可下载的 320×180 YouTube 故事板。正文仅保留经逐张目视确认且核心文字可辨识的画面，复杂界面改用明确标注的原创矢量图。

目录

1 角色迁移：瓶颈从代码产出转向系统设计	2
1.1 旧模型：人的输入决定线性进展	2
1.2 新瓶颈：意图、边界、审查与取舍	2
1.3 速度与价值必须分开判断	3
1.4 全文路线：把能力组织成工程系统	3
1.5 本章小结	3
2 三种工作空间：启动、控制与规模化	4
2.1 命令行：把任务启动与仓库操作放到第一入口	4
2.2 工作 demo：把需求讨论变成可观察反馈	5
2.3 编辑器：从默认入口变成控制面板	5
2.4 GitHub 界面：远程队列与协作交付	5
2.5 职责矩阵与并行前提	5
2.6 本章小结	6
3 治理栈：从仓库规则到可复用技能	6
3.1 第一层：用仓库级规则固定全局边界	6
3.2 第二层：把高频流程封装成技能	7
3.3 两层治理各自解决不同漂移	8
3.4 本章小结	8
4 定制代理：角色、工具边界与编排	8
4.1 从任务配方走向具名执行者	9
4.2 工具权限决定代理能影响什么	9
4.3 代理与技能如何组合	10
4.4 用决策顺序选择治理层	10
4.5 本章小结	11
5 受控委派：把并行能力闭成交付系统	11
5.1 为什么高能力代理仍需要执行约束	11
5.2 CLI 委派：从当前会话生成候选变更	11
5.3 GitHub issue 委派：把待办队列变成异步工作队列	12
5.4 人在回路：把候选结果变成可负责的交付	12
5.5 从单任务试点到可持续规模化	13
5.6 结语：工程大脑负责系统与最终判断	13
5.7 本章小结	14
6 总结与延伸	14

全文核心结论

AI 代理提高了实现并发度，也把工程瓶颈推向意图表达、系统边界、验证证据与批准决策。开发者的关键产物逐渐从单份代码扩展为一套可重复运行的交付系统：仓库规则固定长期约束，可复用流程减少步骤漂移，具名执行角色承担跨步骤判断，工具权限限制影响范围，隔离环境与候选变更承载执行结果，人工批准门保留最终责任。

本文把 23 分钟演讲重组为五个递进问题：角色为何迁移、工作空间如何分工、规则与可复用流程如何治理、具名执行角色如何配置，以及并行任务如何闭合为可负责的交付系统。文中区分演讲事实、产品特定演示与工程推论，使读者无需观看原视频也能独立理解并应用核心方法。

1 角色迁移：瓶颈从代码产出转向系统设计

本节结论

本节结论：AI 代理改变的核心不是工程师是否还写代码，而是人的稀缺注意力应该投向哪里。当实现工作可以由多个助手并行推进时，人的主要职责转向定义意图、划定边界、判断质量与承担交付决策。只有把这些职责固化为护栏机制（guardrails），更快的生成速度才可能形成有效工程产出。

1.1 旧模型：人的输入决定线性进展

传统开发工作可以直观地理解为一由个人持续推动的队列：工程师分析需求、写代码、调试、测试，再开始下一项工作。演讲者在开场把这种状态概括为“100% 在键盘上”，并指出团队的进展往往受单个工程师可投入时间限制。一个人正在实现功能甲，就可能没有时间处理功能乙；是否推进某项功能，直接取决于这个人此刻能否亲自执行。

这类工作方式的优点是上下文集中。实现者通常同时掌握需求、代码细节和验证结果，反馈链路短，责任也容易定位。代价则是明显的机会成本：同一个人不能在同一时刻深度完成多项实现，任务通常只能顺序前进。这里的“线性”描述的是人的执行容量，并不表示传统工程流程必然低效。

识别角色迁移的关键问题

演讲者所说的变化可以用一个问题识别：**当前项目最稀缺的是敲出实现的时间，还是把任务说清楚并判断结果的能力？**若多个代理已经可以同时生成候选实现，后者通常会更快成为瓶颈。

1.2 新瓶颈：意图、边界、审查与取舍

在 00:01:48-00:02:24，演讲者先强调开发者的工作并未消失，随后使用“重心已经移动”来描述角色变化。工程师仍会写代码，只会更谨慎地选择哪些代码值得亲手写。原因在于，助手数量增加后，实现活动可以被拆开并行处理，人的时间却没有同步增加。

这种变化把四项工作推到前台：

- **定义意图：**把“做一个功能”转化为可判断完成与否的目标、约束和验收条件；
- **划定边界：**规定代理可接触的文件、工具、数据与环境，控制失败的影响范围；
- **审查证据：**检查代码、测试、状态检查和行为结果，识别表面完成与实际正确之间的差距；

- **做出取舍：**决定哪项工作优先、哪些风险可接受，以及候选变更是否可以进入交付链路。

据此可推导，代理带来的主要变化是工作分配方式：人从唯一的实现执行者，转为并行系统的设计者和决策者。这个推导不意味着工程师可以脱离代码。相反，代码理解、架构判断和调试经验决定了人能否提出有效约束，并识别“看起来能运行”的结果中隐藏的错误。

1.3 速度与价值必须分开判断

演讲者在 00:02:24–00:02:43 提到早期工具可能生成大量需要重写的代码。他使用的术语应准确理解为 **AI 劣质产出 (AI slop)**：代理快速生成，却在正确性、可维护性或任务贴合度上不足，最终带来返工的代码或内容。这个术语描述的是质量问题，不能泛指所有 AI 生成物。

速度不等于有效产出

生成量不是交付价值的替代指标。并行代理可以同步放大正确实现，也可以同步放大错误假设、重复修改和难以审查的变更。若验证能力和审查容量保持不变，增加并发只会让待判断的候选结果堆积得更快。

护栏机制由规则、权限、测试、验收和批准门共同组成。提示词或项目说明可以提供方向，却只覆盖其中一部分。演讲者把护栏称为当前代理工作的重要组成，原因就在于生成速度越高，偏差扩散的速度也越高。工程系统需要让错误尽早暴露，并让高风险动作在执行前停在明确的批准节点。

因此，评价代理式开发不能只问“写出了多少代码”，还要继续问：任务目标是否满足，测试是否覆盖关键行为，变更是否可维护，审查者能否理解其影响，以及失败能否被局部隔离。视频没有提供生产环境生产率指标，也没有证明某一工具天然满足这些条件；它给出的是工作重心迁移的方向性判断。

1.4 全文路线：把能力组织成工程系统

后续章节将沿着三层结构展开。第一层是工作空间：命令行负责启动与并行，编辑器负责观察与精调，GitHub 界面承接远程队列和协作交付。第二层是治理：仓库级规则提供全局背景，可复用任务配方封装稳定流程，具名执行角色承担更宽范围的判断与工具编排。第三层是交付闭环：任务经过受限执行、可观察反馈、候选变更和人工批准，才进入合并。

这个路线图回应了角色迁移后的关键问题：当实现不再只能由一个人逐行推进，工程师需要设计任务如何进入系统、如何受到约束、如何被检查，以及由谁作出最终判断。演讲者在后段以“斧头到链锯”类比工具能力的跃迁。链锯可以提高处理速度，同时要求更严格的操作边界和技能；这个比喻说明能力放大伴随控制责任，不能用于证明产出质量会自动提高。

1.5 本章小结

读者应带走的判断：开发者的价值正在集中到意图定义、系统设计和最终判断。代码生成变快以后，质量门、权限边界与审查能力会决定这些速度能否转化为价值。下一章据此重新分配三种工作空间的职责，使人的注意力能够覆盖多个并行任务。

2 三种工作空间：启动、控制与规模化

本节结论

本节结论：代理式开发需要把工作分配给三个互补空间：命令行负责快速启动任务和并行推进，编辑器负责观察代码流与精细调整，GitHub 界面负责远程队列、协作状态与候选变更交付。三者形成一条从启动到控制再到规模化的路径；每个并行任务仍需清晰所有权与隔离边界。

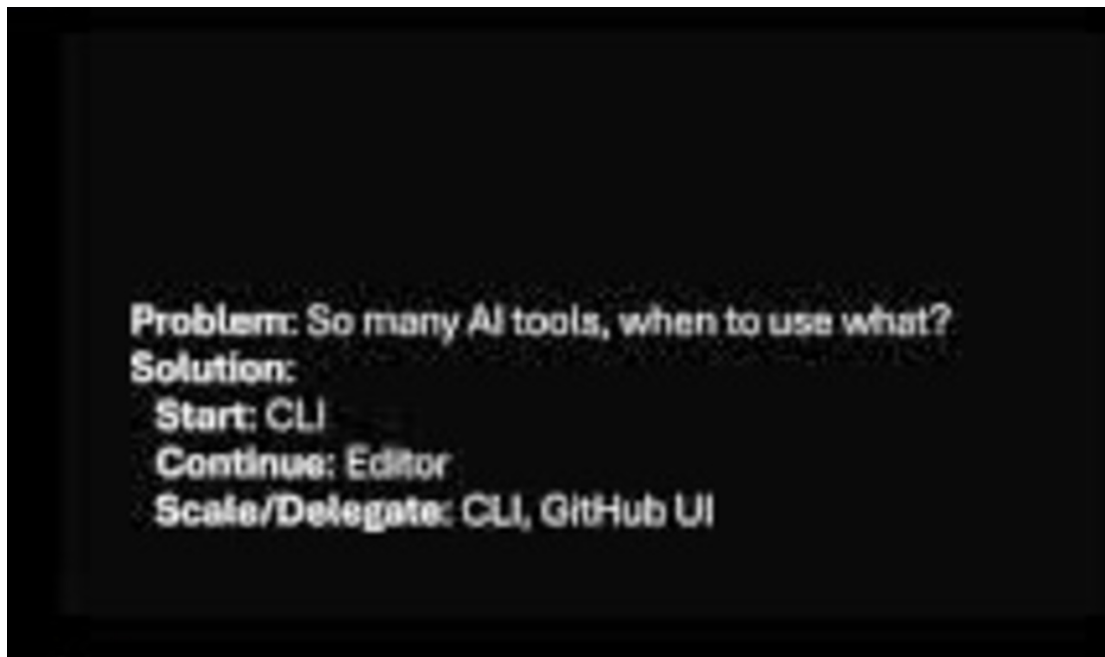


图 1: 演讲者给出的三阶段工作空间：从 CLI 启动，在编辑器继续，通过 CLI 或 GitHub UI 扩展委派。¹

2.1 命令行：把任务启动与仓库操作放到第一入口

在 00:03:16–00:04:34，演讲者提出一个带有个人经验色彩的新工作流：开发者可以从 CLI 开始，再进入编辑器，并通过 CLI 或 GitHub 界面扩展工作规模。他明确说这是自己的方案，听众可以不同意。因此，这一顺序应理解为可检验的工作设计，而不是所有团队都必须遵守的通用标准。

命令行适合作为入口，关键原因是它可以把自然语言任务、仓库操作和执行状态放在同一连续环境中。视频随后列出 issues、pull requests 和 status checks 等使用场景。工程师可以先要求代理生成第一版、修复问题或增加功能，再根据结果决定是否打开编辑器深入处理。入口从编辑器迁移到命令行，省去的是不必要的界面切换，并未省去任务定义和验证。

如何选择工作入口

一个实用的入口判断是：若任务边界清楚、预期输出可验证，命令行适合启动；若需要理解局部行为、比较实现细节或处理模糊交互，编辑器更适合接管。

命令行还降低了启动多个独立任务的操作成本。视频在 00:05:54–00:06:40 展示多个终端分别接收“build an app”“fix an issue”“add a new feature”等指令。这里的重点是任务能够同时运行。画面中

¹ 视频画面时间区间：00:03:27–00:04:33；保留帧约位于 00:03:48。来源为当前网络条件下最高可下载的 320×180 YouTube 故事板。

的终端数量属于演示，不构成最佳并发数。团队真正可用的并发上限取决于隔离环境、审查吞吐、测试速度和任务之间的依赖。

2.2 工作 demo：把需求讨论变成可观察反馈

演讲者在 00:04:53–00:05:19 举了一个简洁例子：一些工程师向业务方带去的已经是 working demo，而不是 PowerPoint，并用 “Is this what you mean?” 发起确认。这个例子真正缩短的是需求澄清回路。可运行的第一版让讨论对象从抽象描述变成可操作行为，利益相关者能够更快指出理解偏差。

工作 demo 仍只是候选答案。它可以验证方向、交互和关键假设，却不能自动证明架构可维护、安全边界完整或生产负载可承受。读者在采用这一做法时，应预先说明 demo 要验证什么，并把未验证事项列出。否则，“可以运行” 容易被误读为 “可以发布”。

从可运行初稿开始确认需求

演讲中的确认方式：先交付一个可工作的初稿，再询问 “这是否是你要表达的意思？” 其教学价值在于让需求反馈围绕具体行为发生，而不是把 demo 当作完成证明。

2.3 编辑器：从默认入口变成控制面板

演讲者有近二十年的编辑器使用经验，并把编辑器称为自己的 “家”。他并没有主张抛弃编辑器。在 00:06:47–00:07:54，他将编辑器重新定位为 control board：开发者可以观察来自 CLI、编辑器本身以及 GitHub 仓库的工作流，在熟悉的代码环境中理解局部结果并进行精调。

“减少在编辑器中的时间” 描述的是职责收缩：当多个代理已经承担成块实现，人无需再次逐行复制其工作。编辑器仍适合处理三类高价值活动：追踪数据或控制流，审查关键差异及其上下文，以及对高风险或高耦合代码进行小范围修改。它还是发现代理是否误解现有架构的重要观察窗口。

编辑器作为控制面板依赖工具能够呈现相关执行流和仓库状态。不同产品、账户层级和企业配置提供的集成程度可能不同。视频只给出一般性描述，读者需要在自己的工具链中确认状态可见性、权限与审计记录，不能假设所有编辑器都能统一接收每一类远程活动。

2.4 GitHub 界面：远程队列与协作交付

在本段中，演讲者主要把 GitHub UI 描述为扩展规模的另一个入口；具体委派流程会在后文展开。对三空间模型而言，GitHub 界面的核心价值是共享状态：issue 可以承载待办边界，pull request 可以承载候选变更，状态检查可以呈现自动验证结果，团队成员也可以异步审查与讨论。

这种职责与命令行相互补充。命令行强调快速启动和连续操作，GitHub 界面强调远程可见性与多人协作，编辑器强调局部理解和精调。某些团队可以通过其他代码托管平台实现同构流程；视频展示的是 GitHub 与 Copilot 语境，不能据此推断每个平台拥有相同的代理能力或命令。

2.5 职责矩阵与并行前提

工作空间	主要职责	适合的问题	关键风险
CLI	启动、批量操作、并行任务	边界清晰且可验证的实现、issue/PR/状态操作	任务互相覆盖、输出过快堆积
编辑器	观察、理解、精调	局部逻辑、高耦合修改、人工审查	重新陷入逐行处理全部任务
GitHub UI	远程队列、协作状态、交付载体	issue 分派、候选变更讨论、状态检查	平台能力依赖、审查队列拥塞

并行工作的前提是隔离与所有权。每个任务应有独立分支、工作目录或隔离执行环境，明确允许修改的范围，并指定谁负责处理冲突和审查结果。共享同一批文件的任务需要先串行化或重新切分。测试也必须能识别跨任务回归；否则多个终端只是把相互干扰隐藏到合并阶段。

并发规模受审查能力约束

并发数不是成熟度指标。可安全运行的任务数量受制于人的审查容量和系统的隔离能力。若审查者只能可靠判断少量变更，就应限制进行中的工作，而不是让代理持续制造新的候选结果。

三空间工作流可以从一个小试点开始：选择边界明确的任务，由 CLI 启动；在编辑器中检查关键实现；将候选变更放入共享审查载体；记录哪些上下文缺失、哪些检查未自动化。只有当任务隔离、测试和审查都稳定后，再增加并行任务。下一章将处理随并发而来的首要矛盾：如何用仓库规则与可复用流程减少输出漂移。

2.6 本章小结

读者应带走的判断：CLI、编辑器与 GitHub 界面分别优化启动、精调和协作规模。合理工作流会让任务在三者之间按风险移动，并用隔离边界、独立所有权和审查容量限制并发。演讲中的多个终端说明并行可能性，不提供可直接复制的最佳数量。

3 治理栈：从仓库规则到可复用技能

本节结论

代理并行能力扩大后，首要治理任务是把项目边界和重复流程写成可复用契约。仓库级规则说明代理所处的工程环境，技能规定某类任务应如何稳定完成；测试、验收与批准门负责提供可执行证据。

3.1 第一层：用仓库级规则固定全局边界

仓库级代理规则文件 `AGENTS.md` 是代理可读取的仓库级指令契约。演讲者把它称为最低限度的护栏，并列出四类信息：仓库意图、应用架构、约束，以及允许和禁止事项。例如，一个项目可以说明采用 React 与 Tailwind，也可以明确“未经授权不得改变架构”。这些内容解决的是上下文漂移：代理每次接手新任务时，都需要知道自己进入了怎样的工程环境。

规则文件应回答什么

- **项目是什么：**目标、用户与核心范围；
- **系统如何组成：**关键模块、依赖方向和不可随意移动的边界；
- **代理负责什么：**允许修改的区域、预期产出与交接方式；
- **哪些行为被禁止：**高风险操作、架构变更条件和必须请求批准的节点；
- **如何证明完成：**测试、静态检查、构建和验收命令。

演讲中的财务追踪应用示例展示了一份较长的 `AGENTS.md`，其中包含项目概览和代理职责。演讲者还展示了一种具体工作法：把另一项目的 `AGENTS.md` 复制到当前项目，再要求代理协助当前代码库适配这些规则。旧规则文件可以作为起点，随后必须由人重新核对当前架构、工具链、禁止事项、风险边界与验证命令；直接照搬会把旧项目假设一并带入新仓库。

规则文件的能力边界

`AGENTS.md` 本身是说明契约，不是机械强制执行引擎。代理可能误读、遗漏或违反说明，因此关键约束仍需由权限配置、测试、持续集成和人工批准门验证。护栏机制正是规则、权限、测试、验收和批准门的组合。

3.2 第二层：把高频流程封装成技能

当问题从“代理不知道项目背景”变成“代理每次都以不同方式完成同一任务”时，需要技能 (Skill)。技能是由代理调用的可复用任务配方，用固定步骤、工具约定和验收条件减少临场发挥。演讲者强调，适合封装的任务通常具备三个特征：频繁发生、可以重复、必须按特定顺序完成。

技能的核心价值可以理解为把工程师脑中的操作清单变成显式契约。例如，发布流程可以规定：先运行测试，再构建产物，随后检查版本和变更记录，任何一步失败都停止并报告。这样的配方让代理知道应使用哪些工具、每一步产出什么，以及何时不得继续。

一个最小技能契约

1. **入口条件：**什么请求触发该技能，哪些前置资源必须存在；
2. **执行步骤：**按顺序列出必要动作，并标明允许使用的工具；
3. **产出契约：**文件、报告或状态必须采用什么形态；
4. **验证方法：**用哪些命令或检查证明结果有效；
5. **失败处理：**何时重试、何时停止、需要向人报告哪些证据。

视频以文件夹中的 `SKILL.md` 为例，说明技能通常带有名称、描述和较长的 Markdown 指令；演讲还展示了通过命令或与名称、描述相符的提示来调用技能。具体目录与调用界面属于产品实现，例如演讲提到的 `.claude/skills`。可迁移的原则是“每个技能自包含、触发条件清楚、步骤和验证明确”，目录位置应以实际平台文档和仓库约定为准。



图 2: 演讲者把技能定义为具名、范围明确、具体且可重复的能力。小字受故事板分辨率限制，正文只采用可辨识的核心定义。²

3.3 两层治理各自解决不同漂移

仓库级规则解决“在什么环境中工作”，技能解决“这类任务怎样做”。前者覆盖整个仓库，后者围绕一个可复用任务；前者提供长期背景和边界，后者提供执行顺序与验收条件。两者共同减少偏差，却都不负责跨多个目标自行规划和协调，这正是第 4 节引入定制代理的原因。

落地时可从一个真实失败案例反推治理内容：若代理经常改变架构，把禁令和批准条件写入仓库级规则；若代理经常漏掉发布步骤，把完整流程封装成技能；若流程涉及跨任务取舍和多工具协调，则需要进一步定义角色、权限和编排。

3.4 本章小结

读者应带走的判断：先用 AGENTS.md 固定仓库意图、架构和禁止事项，再用技能封装高频且顺序明确的任务。说明契约负责引导，权限、测试和批准门负责证明；仅有说明文件无法构成完整护栏。

4 定制代理：角色、工具边界与编排

本节结论

当任务需要跨步骤判断、调用多种技能并协调工具时，应把执行者设计成定制代理。定制代理扩大了能力范围和自主性，同时也扩大了需要治理的风险面；角色、目的和最小工具权限必须一起定义。

²视频画面时间区间：00:10:01–00:12:20；保留帧约位于 00:10:13。来源为当前网络条件下最高可下载的 320×180 YouTube 故事板。

4.1 从任务配方走向具名执行者

定制代理（Custom Agent）是具有明确角色、目的、工具权限与一定规划能力的代理配置。演讲者以安全专家、后端工程师和前端工程师为例：这些名称表达持续角色，而单个技能只表达一个可以重复调用的流程。

演讲把代理称为“高于技能的概念”。这里的“高”指能力范围和自主性更广：代理可以规划、推理、选择工具，并调用多个技能完成目标。它不意味着输出质量天然更高。能力面越宽，越需要清晰的目标、权限限制和验证反馈。

维度	技能	定制代理
核心对象	可重复任务配方	具有目的的执行角色
典型范围	顺序明确的单类流程	跨步骤、跨工具的目标
自主性	按契约执行	规划、推理并选择行动
组合方式	由代理调用	可调用多个技能并协调结果
主要风险	步骤或契约设计不完整	权限过宽、规划偏移、协作冲突

4.2 工具权限决定代理能影响什么

视频展示的定制代理配置除了名称、说明，还包含参数提示与工具字段。工具字段十分关键，因为它决定代理可以读取、搜索、创建待办事项，还是能够修改文件和调用外部服务。演讲者还把使用 MCP 服务器列为定制代理能力范围的一部分：MCP 服务器是代理连接外部工具或数据能力的一种接口，属于工具权限面。开放某个 MCP 连接时，需要同时限定可调用工具、可访问资源、凭据范围和任务目的。权限设计应从任务所需的最小集合开始，而非从平台提供的全部能力开始。

演讲者用研究代理说明这一点：研究任务需要读取、网页搜索和组织信息，通常无需创建或编辑项目文件。关闭写权限可以缩小误操作范围，也使审查者更容易判断其结果属于“研究材料”而非“已落地变更”。如果后续任务确实需要保存报告，可以针对明确目录开放受限写入，而非扩展成全仓写权限。



图 3: 演讲者将规模化描述为让代理承担更多任务，并通过 CLI 或界面完成异步分派；权限边界仍需由工程规则补足。³

最小权限仍需配套验证

限制工具可以降低越界风险，却不能证明研究结论正确，也不能阻止代理在获准范围内产生低质量结果。来源核验、任务验收、运行日志与停止机制仍然是必要依赖。

4.3 代理与技能如何组合

合理的组合方式是让代理负责判断，让技能负责稳定执行。一个“发布协调代理”可以先判断变更是否满足进入发布流程的条件，再调用测试技能、构建技能和发布检查技能；各技能内部保持严格顺序，代理负责根据失败证据决定停止、请求修复或重新运行。

这种分层还能减少重复：测试命令无需复制到每个代理说明中，只需由一个测试技能维护；不同角色在需要时调用同一契约。MCP 服务器作为工具连接层扩展代理可以接触的系统，却不负责替代技能的任务步骤，也不负责替代代理的规划。隐藏依赖必须显式化，例如平台账户、企业功能、外部服务凭据、网络访问和模型能力差异。演讲展示的是 GitHub Copilot 的配置与界面，并仅推测其他平台存在相似概念，因此读者应核对所用产品的实际目录、权限模型和调用方式。

4.4 用决策顺序选择治理层

选择治理层时可以依次询问：

1. 该约束是否对整个仓库长期有效？若是，写入仓库级规则并配套机械检查。
2. 该工作是否高频、可重复且顺序明确？若是，封装为技能。
3. 该目标是否需要跨步骤判断、多工具编排或与其他角色协调？若是，定义定制代理，并收窄其权限。

³视频画面时间区间：00:14:50-00:16:11；保留帧约位于 00:16:10。来源为当前网络条件下最高可下载的 320×180 YouTube 故事板。

4. 结果是否会影响合并、发布或外部系统？若会，保留人工批准门和可停止执行路径。

4.5 本章小结

读者应带走的判断：技能适合稳定配方，定制代理适合具名角色和跨步骤编排。代理的能力范围更广，治理责任也更重；角色、目的、工具权限、输出证据和停止条件应当作为同一份设计来审查。

下一章将把这些角色与权限装入可观察、可审查、可停止的运行闭环，见第 5 节。

5 受控委派：把并行能力闭合成交付系统

本节结论

规模化委派只有进入可观察、可审查、可停止的交付闭环，才会形成工程产能。执行约束框架限定代理如何运行，人在回路保留最终决策权；工程师仍负责设计系统、设置批准门并承担合并与发布判断。

5.1 为什么高能力代理仍需要执行约束

执行约束框架（harness）是包围代理运行过程的控制系统，包括任务边界、工具限制、隔离环境、观察反馈和停止机制。演讲者用“有时像天才、有时犯低级错误”描述代理能力的波动。这种波动意味着并行数量越多，偏差也可能更快扩散。

仓库级规则、技能和定制代理为规模化奠定基础，执行约束框架则把这些静态约定带入运行过程：它规定任务从哪里进入、在哪种环境运行、能够使用什么资源、状态如何呈现、失败时怎样停止，以及结果必须经过谁的批准。

“10 倍、20 倍、100 倍”是修辞

演讲者用这些倍数表达潜在放大效应，视频未提供生产环境指标、对照实验、组织级成本数据或安全模型证明。倍数不能作为预期生产率承诺；真正需要测量的是可接受变更的交付周期、返工率、缺陷率和审查负荷。

5.2 CLI 委派：从当前会话生成候选变更

委派（delegation）是把边界明确的工作交给代理异步执行，同时保留状态观察与结果审查。视频展示了一个 GitHub Copilot 特定流程：用户在 CLI 中输入 `/delegate`，要求创建一个使用 HTML、CSS 和 JavaScript 的支出追踪应用；该命令把会话发送到 GitHub，启动后台任务并形成草稿拉取请求（draft pull request）。草稿 PR 是供检查、讨论和继续修改的候选变更载体，其出现不表示已经达到合并或发布条件。

这个演示有明确前提：当前项目必须是 GitHub 仓库。远程任务状态画面还出现了 Playwright MCP 与 GitHub MCP，说明该次演示通过这些 MCP 连接器调用了浏览器自动化和 GitHub 相关能力；这项画面证据不表示所有委派任务或平台默认提供同等工具。命令名、后台任务形态和界面状态属于 Copilot 产品能力，不能推广成所有代理平台的通用接口。可迁移的工程模式是“本地定义任务—受限工具执行—返回可审查变更”。

受控委派闭环

把并行能力约束为可观察、可审查、可停止的工程交付

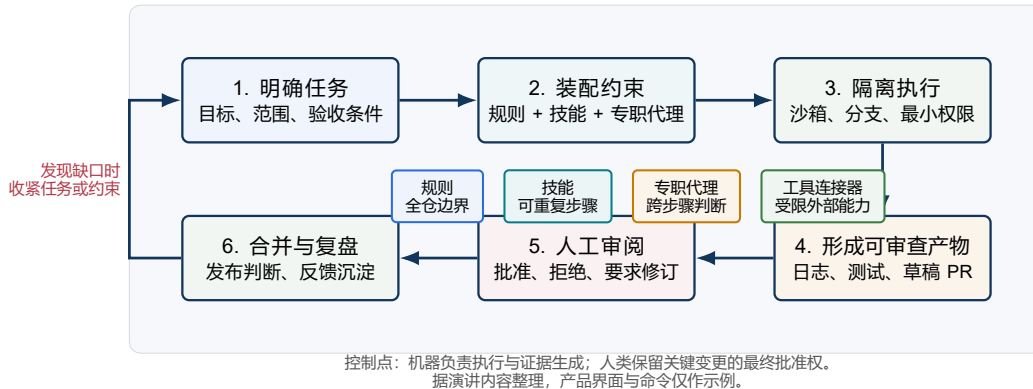


图 4: 受控委派闭环：规则、技能、角色权限和工具连接器共同约束隔离执行，草稿产物经过人工审阅后才进入合并。

代理在沙箱（sandbox）中运行。沙箱限制可访问资源和影响范围，降低任务越界或直接破坏共享环境的风险。它不能替代代码审查、测试和权限设计：一个完全处于沙箱中的代理仍可能生成逻辑错误、不可维护或与需求不符的代码。

5.3 GitHub issue 委派：把待办队列变成异步工作队列

视频展示了第二种 Copilot 特定入口：在 GitHub 仓库界面中创建带标题和描述的 issue，例如“为应用增加深色模式”，随后选择把 issue 分派给代理。演讲者在 00:18:40–00:18:48 明确把这项 GitHub UI 分派限定在拥有 GitHub Enterprise 的情形；这是演讲现场对当时产品条件的陈述，实际可用性仍需依据当前产品方案、组织策略和权限配置核验。界面可以显示任务正在进行，用户能够进入查看状态；代理完成后返回草稿 PR，请求人类判断是否可以交付。

issue 入口适合已有清晰待办队列的团队，但它不会自动修复需求质量。标题和描述过于模糊时，代理只会更快地产生错误方向的候选实现。并行分派还会把审查变成新瓶颈，因此工作进行量必须受审查容量约束；若维护者只能认真审查两项变更，就不应同时启动十项高风险任务。

5.4 人在回路：把候选结果变成可负责的交付

人在回路（human in the loop）意味着人在关键节点拥有查看证据、要求修改、批准、拒绝或停止的权力。仅收到完成通知而无法干预，不满足这一含义。视频中的草稿 PR 是这一控制点：代理提交候选变更，人类检查实现、测试和风险，再决定是否合并。

完整闭环可以整理为：

明确任务 → 规则、技能与代理约束 → 沙箱执行
→ 可观察状态 → 草稿 PR → 人工审查与批准 → 合并

其中每一箭头都有隐藏依赖：任务需要可验收条件；并行工作需要独立分支或隔离目录；测试需要可靠且覆盖关键行为；权限需要最小化；平台账户和企业功能需要可用；审查者需要足够时间理解变更。任何一项缺失，都可能把并行速度转化为返工队列。

5.5 从单任务试点到可持续规模化

读者可以按以下顺序建立自己的委派系统：

1. 选择一个低风险、边界清楚、已有测试的任务作为试点；
2. 在仓库级规则中写清架构边界、禁止事项和验证命令；
3. 把高频验证流程封装成技能，并实际运行其失败路径；
4. 为代理只开放完成任务必需的工具和目录；
5. 要求所有结果以草稿 PR 或等价候选变更交付，保留日志和测试证据；
6. 由人审查需求贴合度、架构影响、安全性与可维护性，再决定合并；
7. 根据审查吞吐和缺陷数据逐步增加并发，发现返工堆积时立即降低工作进行量。

这套顺序把成熟度建立在控制能力上，而不是建立在同时运行多少个代理上。CLI 和 GitHub UI 适合启动、委派和扩展异步工作，编辑器适理解局部代码与精细调整；三种工作空间共同服务于同一交付闭环。

5.6 结语：工程大脑负责系统与最终判断

演讲者用“斧头换成链锯”类比工具带来的速度跃迁，并强调开发者一直在采用更强工具。链锯提升砍伐速度，也要求更严格的操作边界和判断能力。同理，代理扩大代码产量后，工程师的价值集中到设计任务系统、约束代理、设置批准门以及承担发布判断。

演讲最后再次提醒：CLI 与 GitHub UI 可以扩展委派，编辑器更适合精调；“20 倍代码也可能成为 20 倍 AI 劣质产出”。这句话构成全文的质量边界。规模化的目标是扩大经过验证的有效产出，而非扩大代码体积。演讲者的收束信息也十分明确：行业仍然需要“工程大脑”，代理旨在放大工程师，而非取消工程判断。

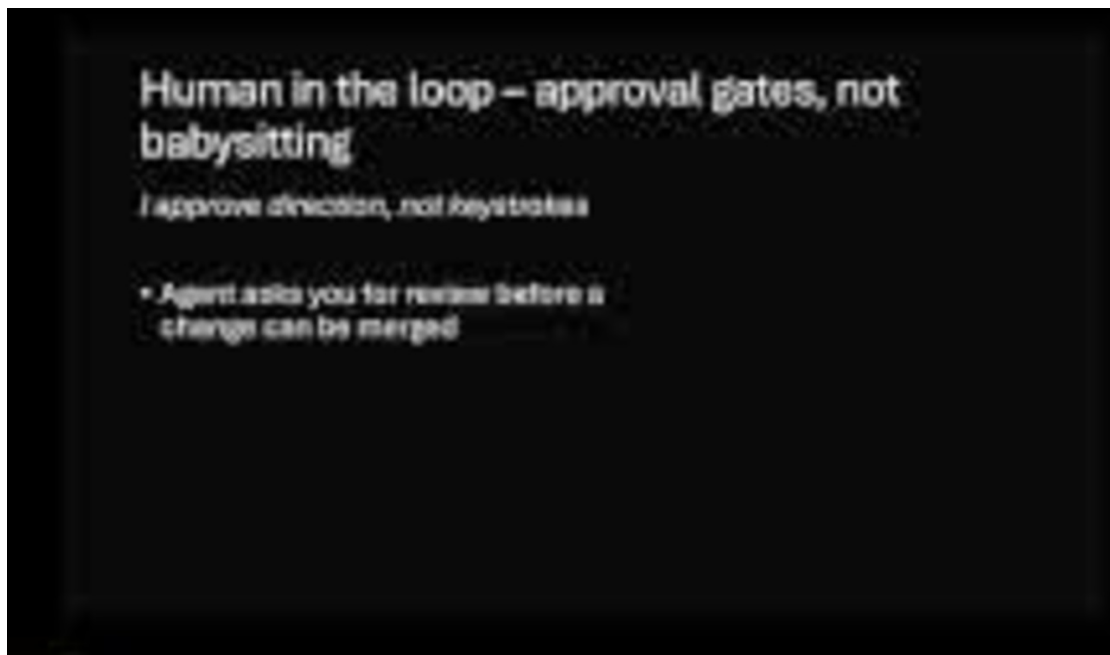


图 5：人在回路应落实为批准门：代理提交候选结果，人类在合并前审阅并保留拒绝权。⁴

来源边界

本章依据演讲中的概念说明与 Copilot 演示整理。视频没有证明这些流程在生产环境中的安全性、收益倍数或组织成本，也没有覆盖各平台的完整权限模型。实际采用前需要以所用产品的当前文档、组织策略和实测结果为准。

5.7 本章小结

读者应带走的判断：委派不转移交付责任。代理可以在受限环境中并行执行并提交候选结果，人类必须保留观察、停止、审查、拒绝和批准权。真正可扩展的开发系统由规则、技能、角色权限、隔离执行、验证证据和人工批准共同组成。

6 总结与延伸

整场演讲可以压缩成一句工程判断：**生成速度越高，系统设计与质量控制越需要显式化**。CLI、编辑器和 GitHub 界面分别承接启动、精调和规模化协作；AGENTS.md、技能与专职代理分别承接环境边界、稳定流程和跨步骤判断；沙箱、验证、草稿 PR 与人工批准门共同把候选代码转化为可负责的交付。

演讲者的“斧头与链锯”类比强调工具会持续增强。它同时暴露出一条容易被忽略的限制：工具放大的是执行能力，执行方向和验收标准仍来自工程判断。所谓 20 倍或 100 倍开发者属于演讲修辞，无法替代返工率、缺陷率、审查负荷、交付周期与事故影响范围等真实指标。

读者可以从一个低风险任务开始验证这套系统：写清验收条件和禁止事项，选择最小权限，要求代理在隔离环境中提交草稿变更，让自动检查与人工审阅共同决定后续行动。若候选结果持续堆积，应先降低并发并修复任务契约、测试或审查流程。规模化的可信依据是可接受变更稳定增加，而非代理数量或生成代码体积增加。

仍需继续追问三个问题：第一，团队如何度量“经过验证的有效产出”；第二，哪些架构决策必须始终由人批准；第三，当多个代理互相依赖时，如何把冲突、来源和责任保留在可审计记录中。这些问题决定代理系统能否从现场演示走向长期工程能力。

⁴视频画面时间区间：00:20:45–00:21:36；保留帧约位于 00:21:06。来源为当前网络条件下最高可下载的 320×180 YouTube 故事板。